

GTU CSE 241 / MIDTERM EXAM

Std Code: 230

Std Number:

2	1	0	1	0	4	0	0	4	0	1	7			
---	---	---	---	---	---	---	---	---	---	---	---	--	--	--

Question 1-A

The main reason is that we use assert for specific cases. If it was global, we could have called this function every single part of our code. Calling from everywhere increases the mistake rate and we could not understand where the specific problem happened.

Even it was usable, I know from you that “we do not like global functions”.

Question 1-B

As you said on lectures, C++ is improved language and we are not writing C anymore. New is easier than malloc thus we can implement the memory allocation more easily and we may prevent memory leaks.

New can we perform everything that malloc can and increases the readability of code.

New is newer, shorter, easier and it was specialized for C++. That is why we use it.

Question 2-A

```
#ifndef POINT2D_H
#define POINT2D_H
#include <iostream>
class Point2D{
    public :
        Point2D( double tx =0.0 , double ty = 0.0 );
        ~Point2D( ) { --count; }
        int number_point( ) const { return count };
        friend ostream& operator<<( ostream& os , const Point2D& );
        double get_x( ) const { return x };
        double get_y( ) const { return y };
        double set_x( double val) { x = val; }
        double set_y( double val) { y = val; }
        bool operator==( const Point2D& ) const;
        bool operator!=( const Point2D&) const;
        Point2D& operator++( );
        Point2D operator++(int ignore);
        Point2D operator!( );
        friend double operator-(const Point2D& , double , double);
    private:
        double x;
        double y;
        static int count;
        double& operator[](int index);
};
#endif
```

Question 2-B

```
#include "Point2D.h"

#include <iostream>

#include <math.h>

using namespace std;

int Point2D :: count = 0;

Point2D :: Point2D( double _x , double _y ) : x( _x ) , y( _y ) {
    count++ ; }

ostream& operator<<( ostream& os , const Point2D& p ) {
    os << p.x << "." << p.y << endl;
    return os; }

bool Point2D :: operator==( const Point2D& p ) const {
    return ( ( x == p.x ) && ( y == p.y ) ) ; }

bool Point2D :: operator!=(const Point2D& p) const {
    return( ( x != p.x ) || ( y != p.y ) ) ; }

Point2D& Point2D :: operator++( ) {
    x++;
    y++;
    return *this ; }

Point2D Point2D :: operator++( int ignore ){
    double tempx = x;
    double tempy = y;
    x++;
    y++;
    return Point2D( tempx , tempy ) ; }
```

```
Point2D Point2D :: operator!( ) {  
    return Point2D( y , x ); } //consructor
```

```
double operator-(const Point2D& p , double x1 , double y1 ) {  
    double dst =0.0;  
    dst = sqrt( pow( abs(x - x1 ) , 2 ) + pow( abs( y - y1 ) , 2 ) );  
    return dst;
```

```
double& point2D :: operator[ ] (int index) {  
    if( index == 0 ) return x;  
    if( index == 1 ) return y; }
```

Question 3 – A

```
#ifndef GTUSET_H
#define GTUSET_H
#include <iostream>
namespace GTU {
    class GTUSet {
    public:
        GTUSet( );
        void insert( int num );
        void erase( int num );
        const operator+( const GTUSet& g ) const;
        bool find ( int num ) const ;
        friend std :: ostream& operator<< (std :: ostream& , const GTUSet& ) ;
        void increase_cpt( );
        GTUSet ( const GTUSet& obj ) ; //not used
        GTUSet operator^ ( const GTUSet& g ) const;
        ~GTUSet( ) { delete [ ] data ; data = nullptr ; }
        //no place for deconsructor , I am sorry )

    private :
        int * data ;
        int capacity;
        int size;
    };
}
#endif
```

QUESTION 3-B

```
#include "GTUSet.h"
```

```
#include <iostream>
```

```
namespace GTU {
```

```
    GTUSet :: GTUSet ( int size( 0 ) , int capacity( 10 ) , int* data( nullptr ) ) {  
        data = new int[capacity] ; }  
}
```

```
void GTUSet :: increase_cpt ( ) {  
    capacity *= 2;  
    int* temp = new int[capacity];  
    for( int i = 0 ; i < size ; i++ ) {  
        temp[i] = data[i]; }  
    delete [ ] data;  
    data = temp ; }  
}
```

```
void GTUSet :: insert( int num ) {  
    if(size +1 >= capacity) increase_cpt( )  
    data[size++] = num ; }  
}
```

```
void GTUSet :: erase( int num ) {  
    int flag = 0 ;  
    int k = 0 ;  
    while( k < size ) {  
        if(num == data[k]) {  
            flag = 1 ; }  
        k++ ; }  
    if( !flag ) return ;  
}
```

```

GTUSet temp;
for( int i = 0 ; i < size ; i++ ) {
    if(num != data[i] ) temp.insert( data[i] );
}
delete [ ] data ;
size = temp.size ;
data = temp.data; }

```

```

std::ostream operator <<( std::ostream& os , const GTUSet& g ){
    for( int i = 0 ; i < size ; i++ ){
        os << data[i] << “ , “ ; }
    return os; }

```

```

const GTUSet GTUSet ::operator+ (const GTUSet& g ) const {
    GTUSet union;
    int flag;
    for( int i = 0 ; i < size ; i++ ) {
        union.insert( data[i] ) ; }
    for( int j = 0 ; j < g.size ; j++ ) {
        flag = 0;
        for(int k =0 ; k< union.size ; k++ ) {
            if(union.data[k] == g.data[j])
                flag = 1 ; break
        }
        if( !flag) {

```

```

        inter.insert( g.data[ j] );
    }
    return union; }

```

```

const GTUSet GTUSet :: operator^ (const GTUSet& g) const {
    GTUSet inter;
    for( int i = 0 ; i < size ; i++ ){
        inter.insert( data[i] ) ; }

    for( int j = 0 ; j < g.size ; j++ ){
        for( int k = 0 ; k < inter.size ; k++ ) {
            if( inter.data[k] != g.data[j] ) {
                inter.erase(inter.data[k] );
            }
        }
    }
    return inter; }

```

```

bool GTUSet :: find ( int num ) {
    for(int i = 0 ; i < size ; i++ ){
        if(data[i] = num ) return true; }
    return false ; }

} // namespace

```


